🏋️ Warsztat: Widoki, Widoki Zmaterializowane i Tabele Tymczasowe w PostgreSQL 17

🎯 Cele warsztatu

- Poznasz różnice między widokiem, widokiem zmaterializowanym a tabelą tymczasową.
- Zobaczysz, jak je tworzyć, wykorzystywać i łączyć ze sobą.
- Nauczysz się rozpoznawać, kiedy które rozwiązanie jest najlepsze.

🔹 1. Przygotowanie środowiska

Utwórzmy tabele i dane przykładowe (symulacja systemu sprzedażowego):

```
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name        TEXT,
    city        TEXT
);

CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INT REFERENCES customers(customer_id),
    order_date   DATE,
    total_amount NUMERIC(10,2)
);

INSERT INTO customers (name, city) VALUES
('Jan Kowalski', 'Warszawa'),
('Anna Nowak', 'Kraków'),
('Piotr Iksiński', 'Gdańsk');

INSERT INTO orders (customer_id, order_date, total_amount) VALUES
(1, '2025-09-01', 120.50),
(1, '2025-09-02', 350.00),
(2, '2025-09-03', 200.00),
(3, '2025-09-03', 560.00),
(2, '2025-09-04', 140.00);
```

🔹 2. Ćwiczenie z widokami

1. **Prosty widok łączący klientów i zamówienia**:

```
SELECT * FROM customer_orders;
```

	name	city	order_id	total_amount
1	Jan Kowalski	Warszawa	1	120.5
2	Jan Kowalski	Warszawa	2	350
3	Anna Nowak	Kraków	3	200
4	Piotr Iksiński	Gdańsk	4	560
5	Anna Nowak	Kraków	5	140

2. **Widok filtrujący zamówienia > 200 zł**:

```
select * from orders_above_200
```

	order_id	customer_id	order_date	total_amount
1	2	1	2025-09-02	350
2	4	3	2025-09-03	560

▶️ Pytanie: czy możesz wykonać INSERT do tego widoku?

3. **Widok „maskujący" dane** (np. ukrywający kwotę sprzedaży):

```
select * from customer_orders_masked
```

	name	order_id	total_amount
1	Jan Kowalski	1	[ukryto]
2	Jan Kowalski	2	[ukryto]
3	Anna Nowak	3	[ukryto]
4	Piotr Iksiński	4	[ukryto]
5	Anna Nowak	5	[ukryto]

---

3. Ćwiczenie z widokami zmaterializowanymi

1. **Raport sprzedaży dziennej**:

```
select * from daily_sales_summary
```

	order_date	daily_sales
1	2025-09-01	120.5
2	2025-09-02	350
3	2025-09-03	760
4	2025-09-04	140

2. **Dodaj nowe zamówienie i odśwież dane**:

```
select * from daily_sales_summary
```

	order_date	daily_sales
1	2025-09-01	120.5
2	2025-09-02	350
3	2025-09-03	760
4	2025-09-04	140
5	2025-09-05	500

4. Ćwiczenie z tabelami tymczasowymi

1. **Stwórz tabelę tymczasową dla zamówień po 2.09.2025**:

```
select * from temp_recent_orders
```

	order_id	customer_id	order_date	total_amount
1	3	2	2025-09-03	200
2	4	3	2025-09-03	560
3	5	2	2025-09-04	140
4	6	1	2025-09-05	500

2. **Eksperymentuj z danymi** (np. prognoza wzrostu o 10%):

```
select * from temp_recent_orders
```

	order_id	customer_id	order_date	total_amount
1	3	2	2025-09-03	220
2	4	3	2025-09-03	616
3	5	2	2025-09-04	154
4	6	1	2025-09-05	550

3. **Sprawdź żywotność**:
   - zamknij sesję i otwórz nową konsolę → spróbuj zapytać temp_recent_orders.

5. Integracja – wszystkie elementy

1. Utwórz widok customer_orders (już istnieje z kroku 2).
2. Na jego podstawie zbuduj **widok zmaterializowany**:

```
select * from sales_by_city
```

	city	total_sales
1	Warszawa	970.5
2	Kraków	340
3	Gdańsk	560

3. W swojej sesji przygotuj **tabelę tymczasową** tylko dla Warszawy:

```
select * from temp_warsaw_sales
```

	city	total_sales
1	Warszawa	970.5

4. Eksperyment: sprawdź, jak zmienia się sprzedaż w Warszawie przy +15% i wynik zapisz w nowej tabeli tymczasowe:

```
select * from temp_warsaw_sales_plus_15
```

	city	total_sales	forecast
1	Warszawa	970.5	1,116.075